MEMORY MANAGEMENT



DR NISHANT JAIN
MITS-DU GWALIOR

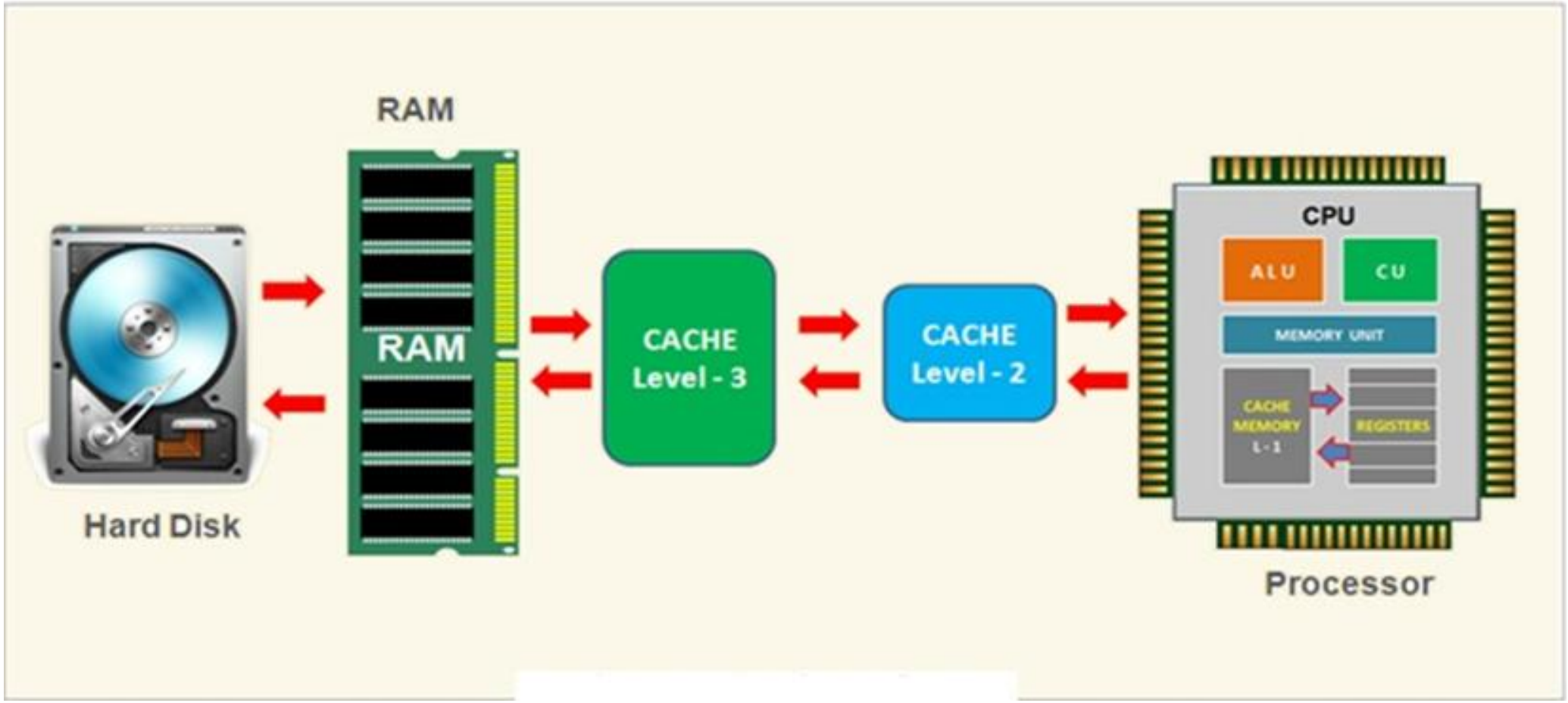
DR.NISHANT JAIN, MITS-DU GWLIOR

INTRODUCTION

- Memory is the important part of the computer that is used to store the data.
- Its management is critical to the computer system because the amount of main memory available in a computer system is very limited.
- At any time, many processes are competing for it.
- Moreover, to increase performance, several processes are executed simultaneously.
- For this, we must keep several processes in the main memory, so it is even more important to manage them effectively.

DR NISHANT JAIN,
MITS-DU GWLIOR

BASICS OF MEMORY MANAGEMENT



DR. NISHANT JAIN
MEMORY MANAGEMENT
MITS-DU GWALIOR

CACHE MEMORY

- **Cache memory is a chip-based computer component that makes retrieving data from the computer's memory more efficient.**
- **It acts as a temporary storage area that the computer's processor can retrieve data from easily.**
- **It is more readily available to the processor than the computer's main memory. IT is also called the CPU memory.**

PRASHANT JAIN
MEMORY MANAGEMENT
MITS-DU GWALIOR

TYPES OF CACHE MEMORY

- **L1 cache, or primary cache, is extremely fast but relatively small, and is usually embedded in the processor chip as CPU cache.**
- **L2 cache, or secondary cache, is often more capacious than L1. L2 cache may be embedded on the CPU, or it can be on a separate chip or coprocessor and have a high-speed alternative system bus connecting the cache and CPU.**
- **Level 3 (L3) cache is specialized memory developed to improve the performance of L1 and L2. L1 or L2 can be significantly faster than L3, though L3 is usually double the speed of DRAM. With multicore processors, each core can have dedicated L1 and L2 cache, but they can share an L3 cache. If an L3 cache references an instruction, it is usually elevated to a higher level of cache.**

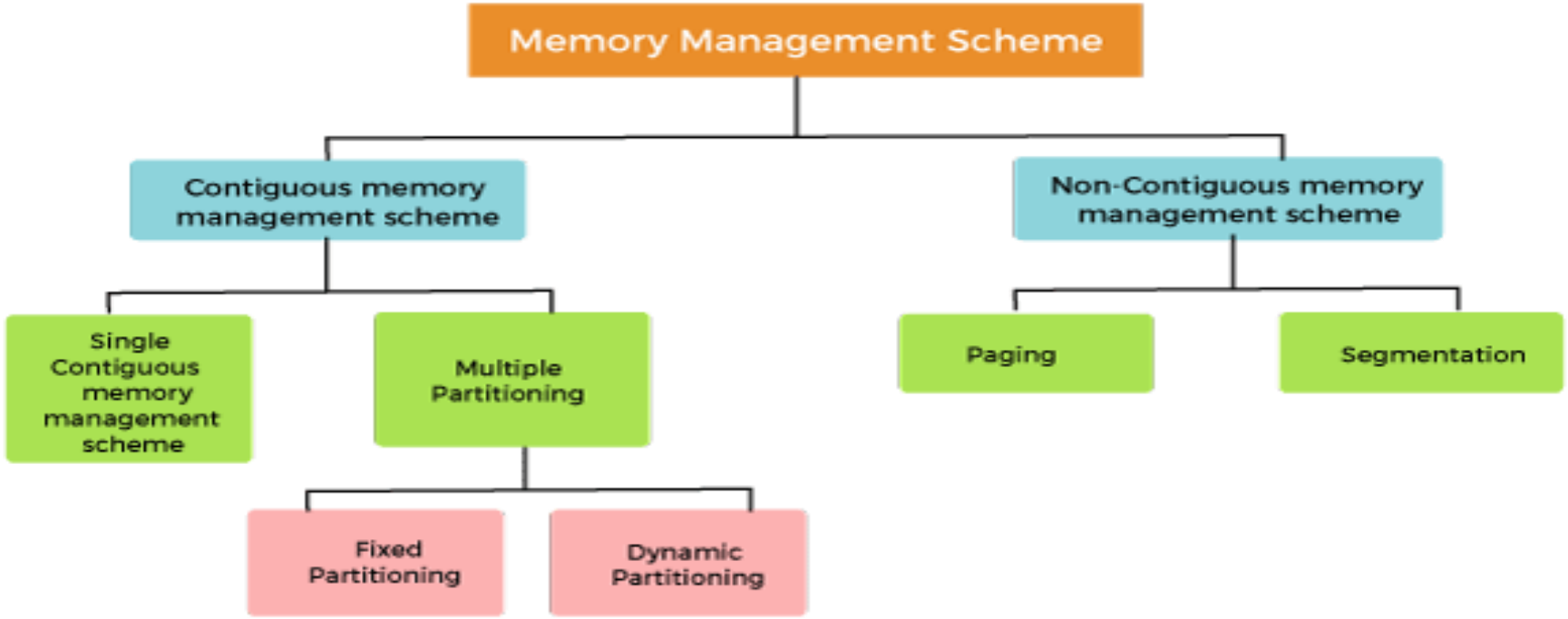
ROLE OF MEMORY MANAGER

- Memory manager is used to keep track of the status of memory locations, whether it is free or allocated.
- Memory manager permits computers with a small amount of main memory to execute programs larger than the size or amount of available memory.
- The memory manager is responsible for protecting the memory allocated to each process from being corrupted by another process.
- Memory managers should enable sharing of memory space between processes. Thus, two programs can reside at the same memory location although at different times.

MEMORY MANAGEMENT TECHNIQUES

The memory management techniques can be classified into following main categories:

- Contiguous memory management schemes
- Non-Contiguous memory management schemes



Classification of memory management schemes

CONTIGUOUS MEMORY MANAGEMENT SCHEME

Single contiguous memory management scheme

- **The Single contiguous memory management scheme is the simplest memory management scheme used in the earliest generation of computer systems.**
- **In this scheme, the main memory is divided into two contiguous areas or partitions.**

The operating systems reside permanently in one partition, generally at the lower memory, and the user process is loaded into the other partition.

Single contiguous memory management scheme

Advantages:

- **Simple to implement.**
- **Easy to manage and design.**
- **In a Single contiguous memory management scheme, once a process is loaded, it is given full processor's time, and no other processor will interrupt it.**

Disadvantages:

- **Wastage of memory space due to unused memory as the process is unlikely to use all the available memory space.**
- **The CPU remains idle, waiting for the disk to load the binary image into the main memory.**
- **It can not be executed if the program is too large to fit the entire available main memory space.**
- **It does not support multiprogramming, i.e., it cannot handle multiple programs simultaneously.**

CONTIGUOUS MEMORY MANAGEMENT SCHEME

Multiple Partitioning

The single Contiguous memory management scheme is inefficient as it limits computers to execute only one program at a time resulting in wastage in memory space and CPU time. The problem of inefficient CPU use can be overcome using multiprogramming that allows more than one program to run concurrently.

To switch between two processes, the operating systems need to load both processes into the main memory. The operating system needs to divide the available main memory into multiple parts to load multiple processes into the main memory. Thus multiple processes can reside in the main memory simultaneously.

PRASHANT JAIN
MITS-DU GWALIOR

Multiple Partitioning

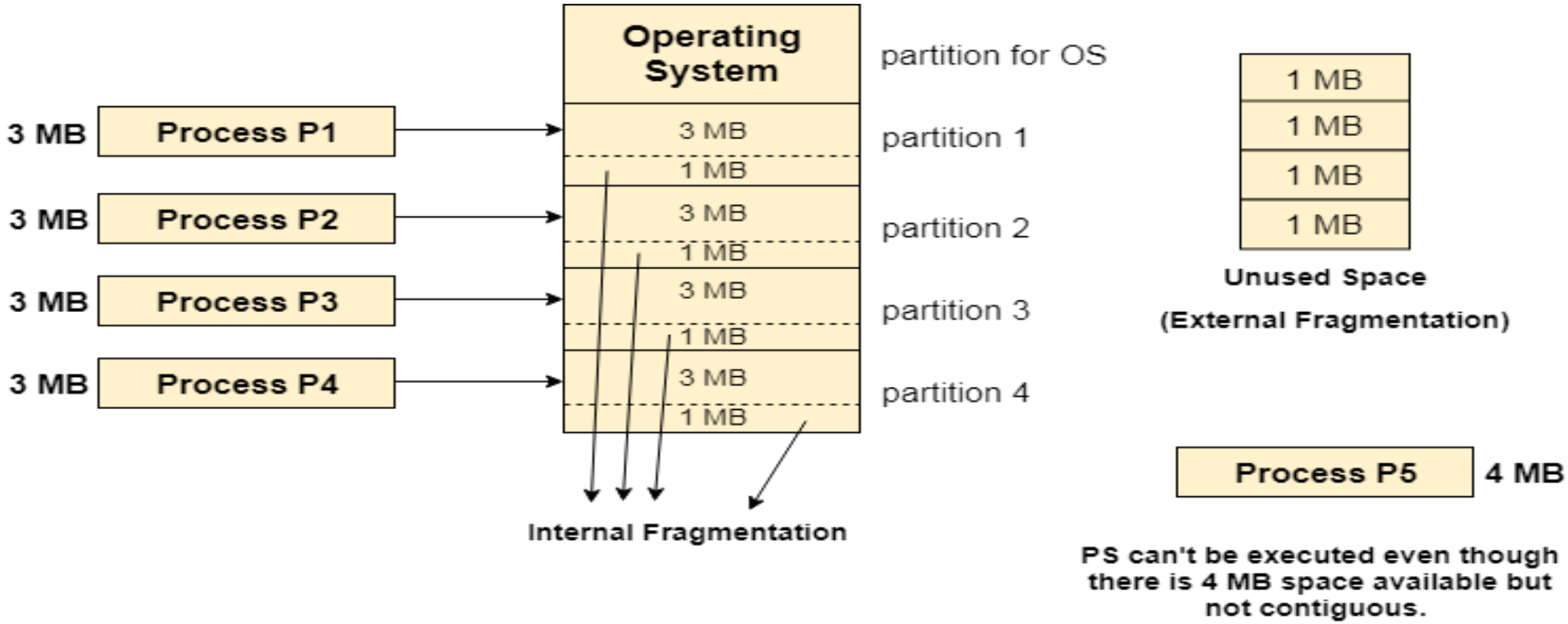
Fixed Partitioning

- The main memory is divided into several fixed-sized partitions in a fixed partition memory management scheme or static partitioning.
- These partitions can be of the same size or different sizes.
- Each partition can hold a single process.
- The number of partitions determines the degree of multiprogramming, i.e., the maximum number of processes in memory.
- These partitions are made at the time of system generation and remain fixed after that.

PRASHANT JAIN
MEMORY MANAGEMENT
MITS-DU GWALIOR

Multiple Partitioning

Fixed Partitioning



Fixed Partitioning

(Contiguous memory allocation)

DR. NISHANT JAIN
MEMORY MANAGEMENT
MITS-DU GWALIOR

Multiple Partitioning

Fixed Partitioning

Advantages:

- **Simple to implement.**
- **Easy to manage and design.**

Disadvantages:

- **This scheme suffers from internal fragmentation.**
- **The number of partitions is specified at the time of system generation.**

Multiple Partitioning

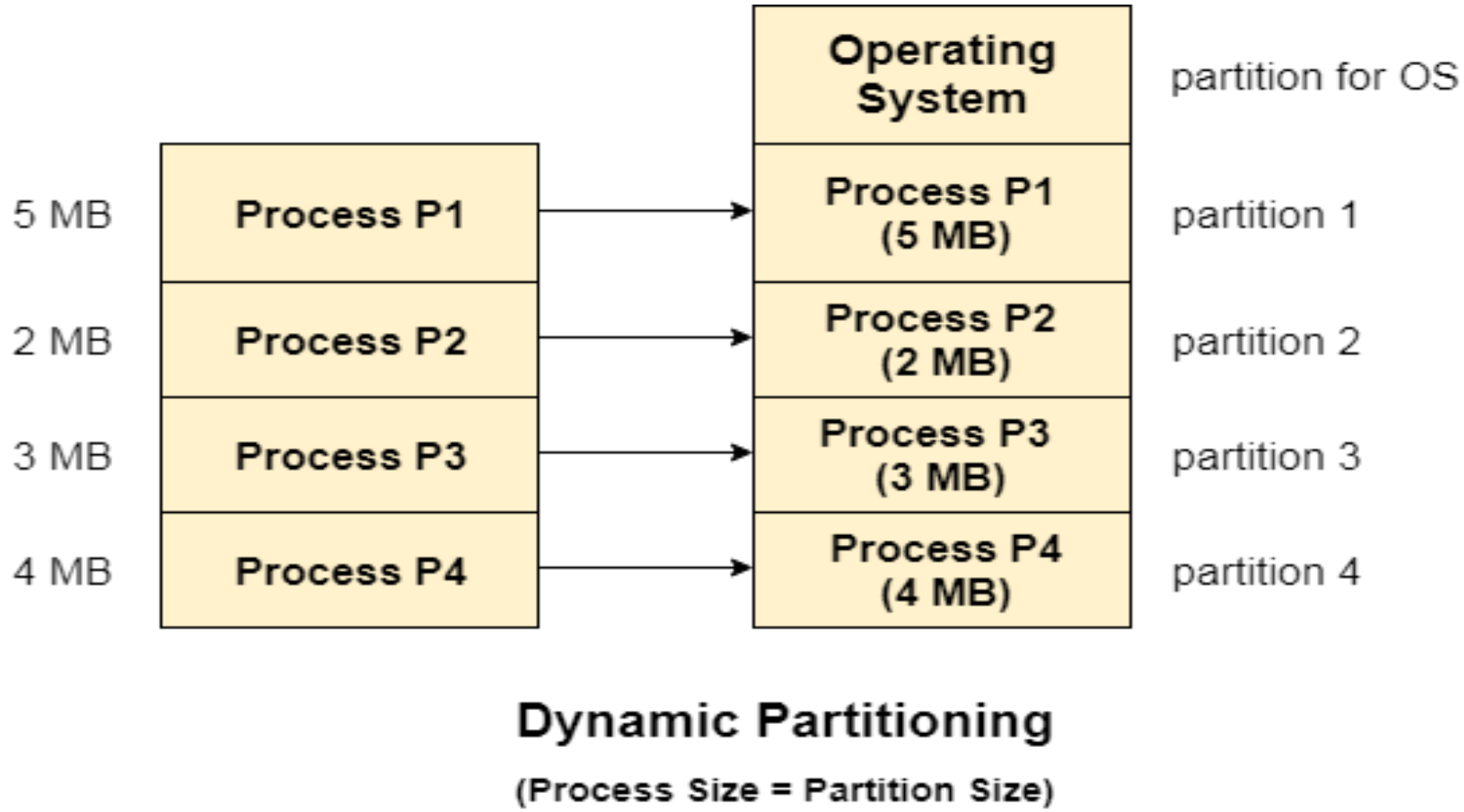
Dynamic Partitioning

- The dynamic partitioning was designed to overcome the problems of a fixed partitioning scheme.
- In a dynamic partitioning scheme, each process occupies only as much memory as they require when loaded for processing.
- Requested processes are allocated memory until the entire physical memory is exhausted or the remaining space is insufficient to hold the requesting process.
- In this scheme the partitions used are of variable size, and the number of partitions is not defined at the system generation time.

PRASHANT JAIN
MITS-DU GWALIOR

Multiple Partitioning

Dynamic Partitioning



Dynamic Partitioning

(Process Size = Partition Size)

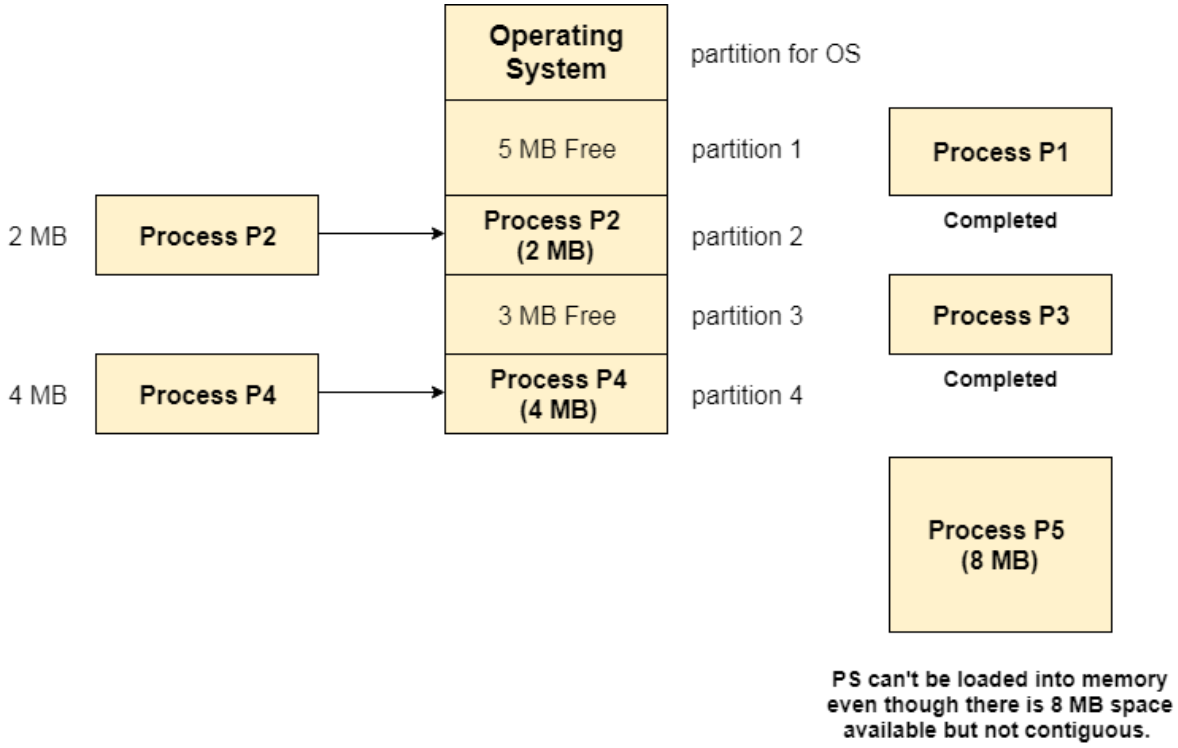
DR. NISHANT JAIN
MEMORY MANAGEMENT
MITS-DU GWALIOR

Multiple Partitioning

Dynamic Partitioning

Advantages:

- Simple to implement.
- No Internal Fragmentation
- No Limitation on the size of the process
- Degree of multiprogramming is dynamic
- Easy to manage and design.



External Fragmentation in Dynamic Partitioning

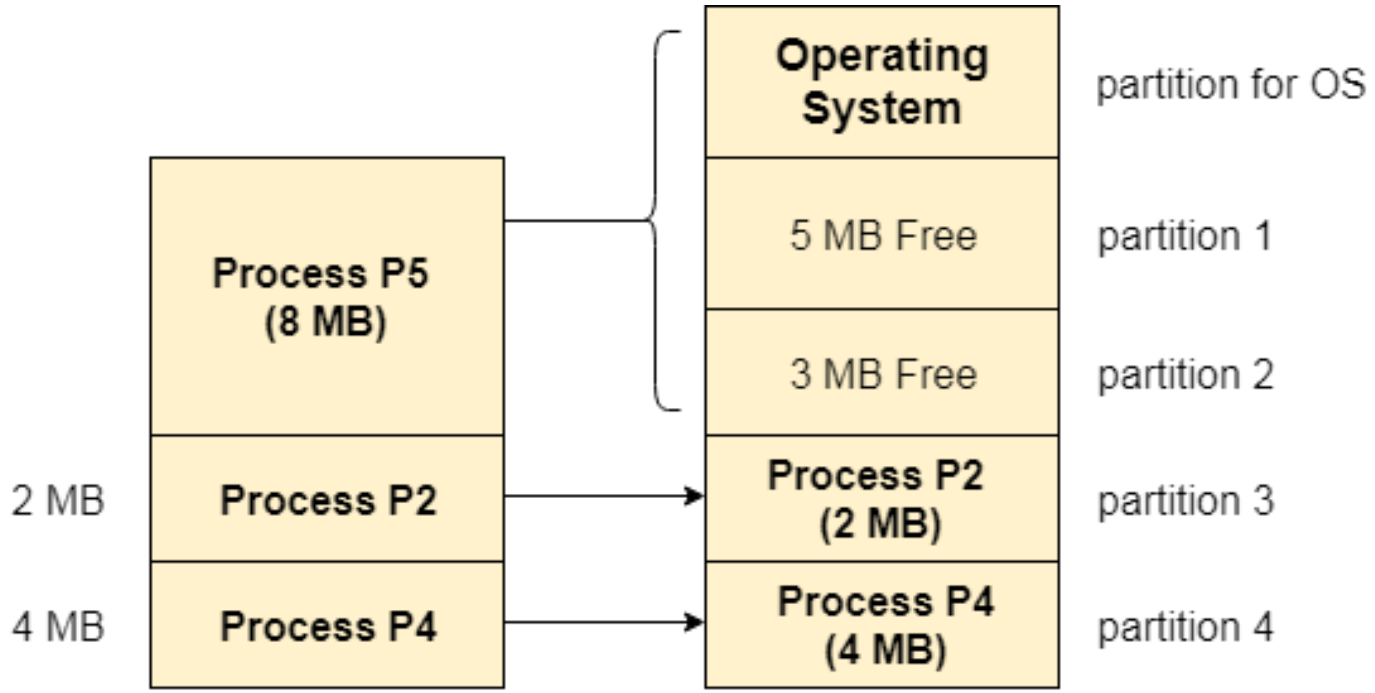
X

Disadvantages:

- This scheme also suffers from external fragmentation.
- The number of partitions is specified at the time of system segmentation.

DR NISHANT JAIN,
MITS-DU GWLIOR

COMPACTION



Now PS can be loaded into memory because the free space is now made contiguous by compaction

Compaction

DR NISHANT JAIN,
MITS-DU GWLIOR

COMPACTION

Problem with Compaction

The efficiency of the system is decreased in the case of compaction due to the fact that all the free spaces will be transferred from several places to a single place.

Huge amount of time is invested for this procedure and the CPU will remain idle for all this time. Despite of the fact that the compaction avoids external fragmentation, it makes system inefficient.

1 B transfer needs 6 NS

256 MB transfer needs 256 X 2^20 X 6 X 10 ^ -9 secs

ALLOCATION ALGORITHM

DR NISHANT JAIN,
MITS-DU GWLIOR

1. First Fit Algorithm

2. Next Fit Algorithm

3. Best Fit Algorithm

4. Worst Fit Algorithm

5. Quick Fit Algorithm

The first fit algorithm is **the best algorithm** among all because

1. It takes lesser time compare to the other algorithms.
2. It produces bigger holes that can be used to load other processes later on.
3. It is easiest to implement.

DYNAMIC STORAGE-ALLOCATION PROBLEM

DR NISHANT JAIN,
MITS-DU GWLIOR

1. **Given memory partitions of 100 KB, 500 KB, 200 KB, 300 KB, and 600 KB, how would each of the first fit, best fit and worst fit algorithms place processes of 212 KB, 417 KB, 112 KB and 426 KB? Which algorithm makes the most efficient use of memory?**

   Ans: a) First-fit

   The first-fit algorithm selects the first free partition that is large enough to accommodate the request.

   First-fit would allocate in the following manner:

   212 KB => 500 KB partition, leaves a 288 KB partition

   417 KB => 600 KB partition, leaves a 183 KB partition

   112 KB => 288 KB partition, leaves a 176 KB partition

   426 KB would not be able to allocate, no partition large enough!

DYNAMIC STORAGE-ALLOCATION PROBLEM

DR NISHANT JAIN,
MITS-DU GWLIOR

b) Best-fit

The best-fit algorithm selects the partition whose size is closest in size (and large enough) to the requested size.

Best-fit would allocate in the following manner:

212 KB => 300 KB, leaving a 88 KB partition

417 KB => 500 KB, leaving a 83 KB partition

112 KB => 200 KB, leaving a 88 KB partition

426 KB => 600 KB, leaving a 174 KB partition

DYNAMIC STORAGE-ALLOCATION PROBLEM

DR NISHANT JAIN,
MITS-DU GWLIOR

c) Worst-fit

The worst-fit algorithm effectively selects the largest partition for each request.

Worst-fit would allocate in the following manner:

212 KB => 600 KB, leaving a 388 KB partition

417 KB => 500 KB, leaving a 83 KB partition

112 KB => 388 KB, leaving a 276 KB partition

426 KB would not be allowed to allocate as no partition is large enough!

Which algorithm makes the most efficient use of memory?

The best-fit algorithm performed the best of the three algorithms, as it was the only algorithm to meet all the memory requests.

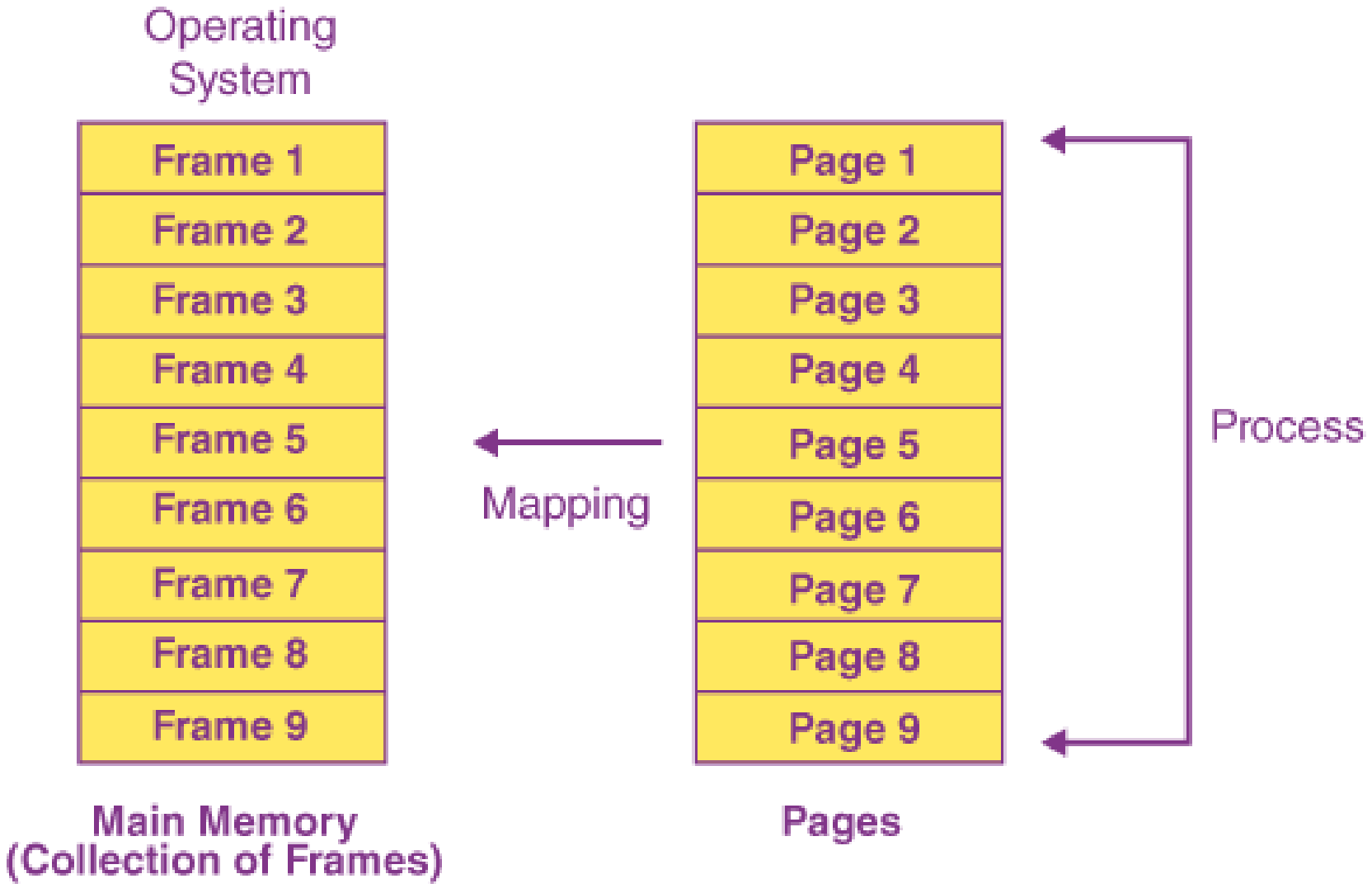
PAGING

What is Paging in the OS?

- Paging is a storage mechanism used in OS to retrieve processes from secondary storage to the main memory as pages.
- The primary concept behind paging is to break each process into individual pages. Thus the primary memory would also be separated into frames.
- One page of the process must be saved in one of the given memory frames. These pages can be stored in various memory locations, but finding contiguous frames/holes is always the main goal.
- Process pages are usually only brought into the main memory when they are needed; else, they are stored in the secondary storage.
- The frame sizes may vary depending on the OS. Each frame must be of the same size.
- Since the pages present in paging are mapped on to the frames, the page size should be similar to the frame size

PRASHANT JAIN
MITS-DU GWALIOR

PAGING



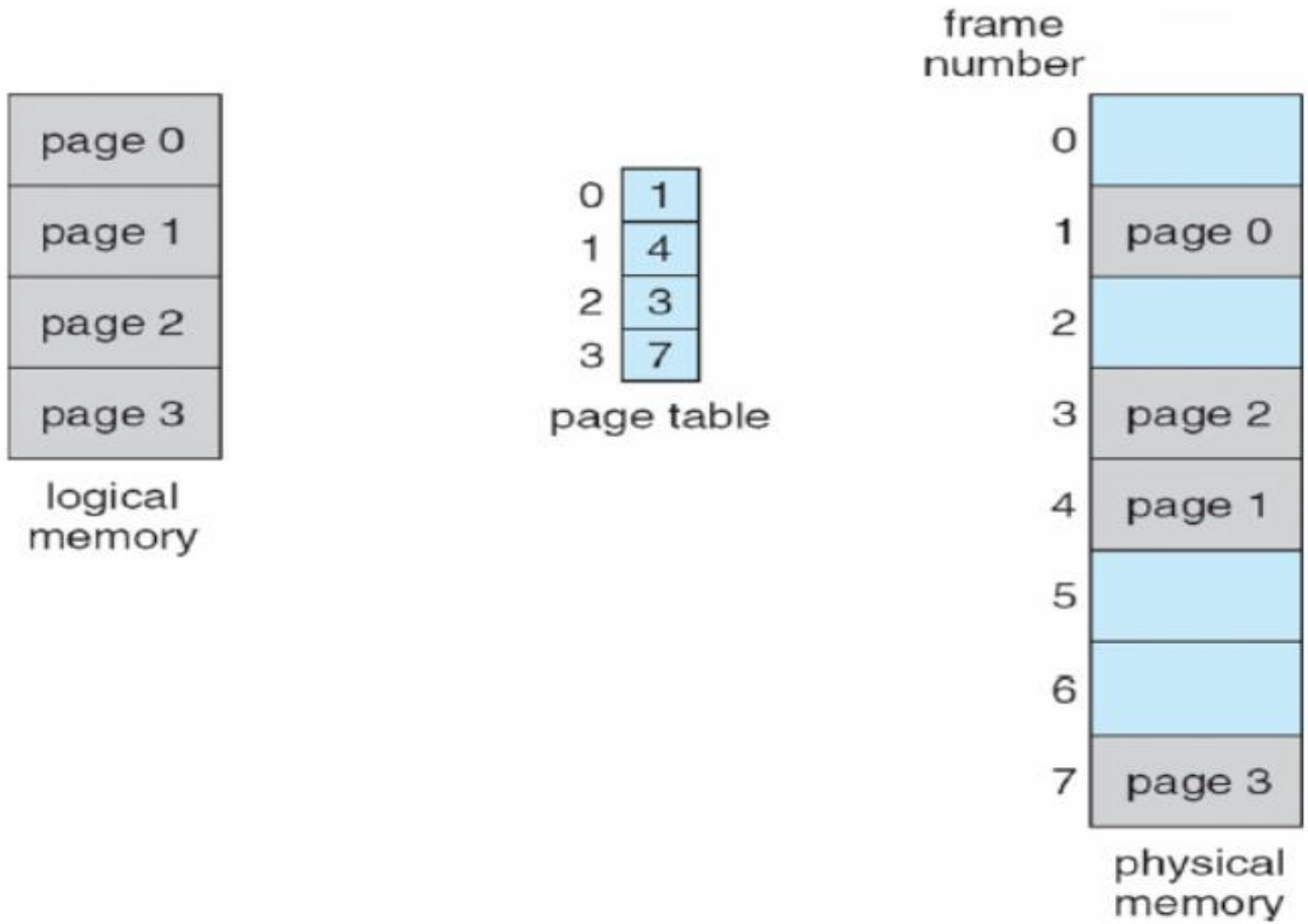
MEMORY MANAGEMENT UNIT

DR NISHANT JAIN, PACE MITS-DU GWLIOR

- The Memory Management Unit (MMU) is responsible for **converting logical addresses to physical addresses**.
- The physical address refers to the actual address of a frame in which each page will be stored, whereas the logical address refers to the address that is generated by the CPU for each page.
- When the CPU accesses a page using its logical address, the OS must first collect the physical address in order to access that page physically.
- There are two elements to the logical address:
  - Page number
  - Offset
- The OS's memory management unit must convert the page numbers to the frame numbers.



Paging Model of Logical and Physical Memory



MAPPING FROM PAGE TABLE TO MAIN MEMORY

DR NISHANT JAIN,
PACE
MITS-DU GWLIOR

1. Generation of logical address

CPU generates logical address for each page of the process. This contains two parts: page number and offset.

2. Scaling

Each time the address is generated, the value of the page table base is added to the page number to get the actual location of the page entry in the table. This process is called scaling.

3. Generation of physical Address

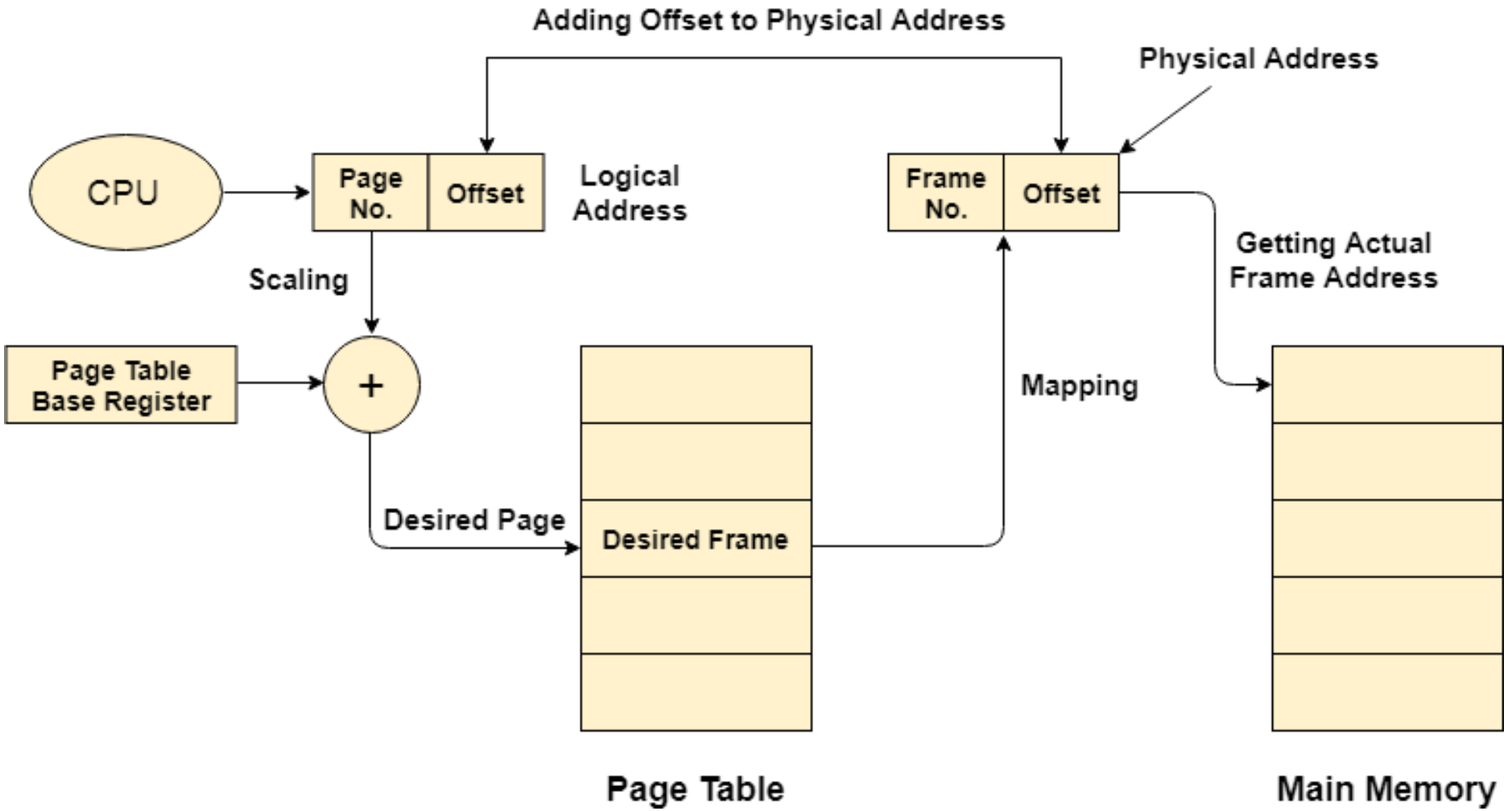
The frame number of the desired page is determined by its entry in the page table. A physical address is generated which also contains two parts : frame number and offset. The Offset will be similar to the offset of the logical address therefore it will be copied from the logical address.

4. Getting Actual Frame Number

The frame number and the offset from the physical address is mapped to the main memory in order to get the actual word address.

MAPPING FROM PAGE TABLE TO MAIN MEMORY

DR. NISHANT JAIN,
PACE,
MITS-DU GWLIOR



MAPPING FROM PAGE TABLE TO MAIN MEMORY

DR NISHANT JAIN,
PAGNIS
MITS-DU GWLIOR

The address generated by the CPU is divided into the following:

- **Page offset(d):** It refers to the number of bits necessary to represent a certain word on a page, page size in Logical Address Space, or page word number or page offset.
- **Page number(p):** It is the number of bits needed to represent the pages in the Logical Address Space or the page number.

The Physical Address is divided into the following:

- **Frame offset(d):** It refers to the number of bits necessary to represent a certain word in a frame, or the Physical Address Space frame size, the word number of a frame, or the frame offset.
- **Frame number(f):** It's the number of bits needed to indicate a frame of the Physical Address Space or a frame number.

MAPPING FROM PAGE TABLE TO MAIN MEMORY

DR NISHANT JAIN, MITS-DU GWLIOR PAGING

Physical Address Space = M words

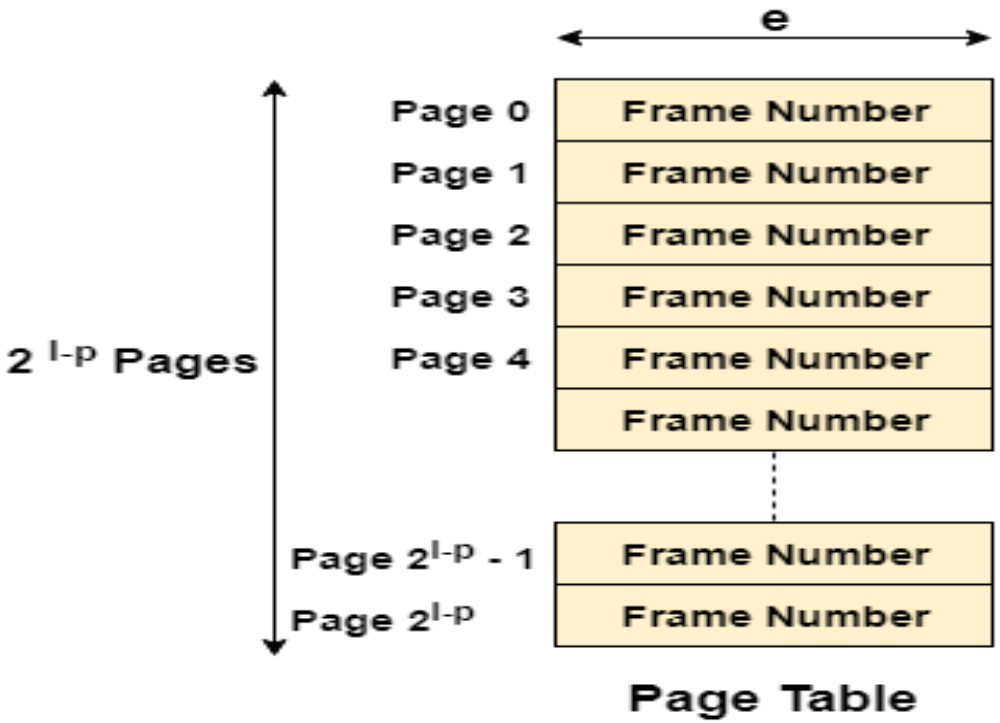
Logical Address Space = L words

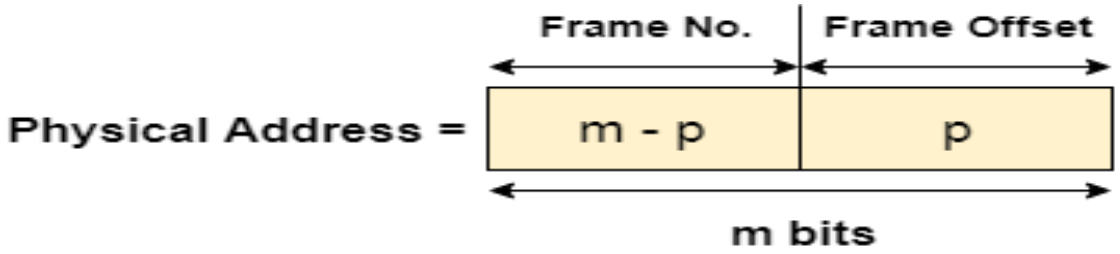
Page Size = P words

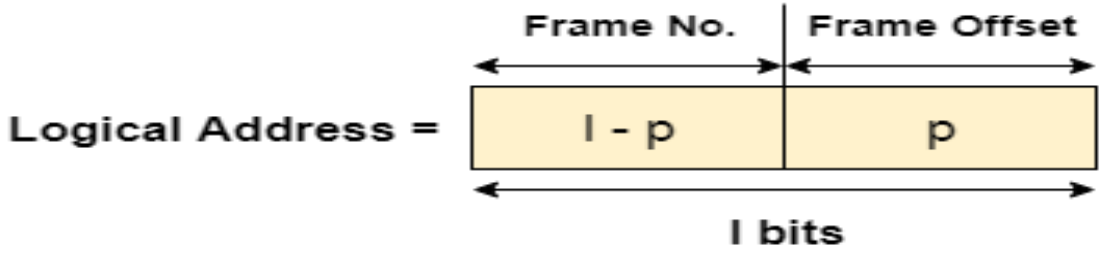
Physical Address = $\log_2 M$ = m bits

Logical Address = $\log_2 L$ = l bits

page offset = $\log_2 P$ = p bits







No. of entries in Page Table = No. of the pages in the process

Page Table Size = 2^{l-p} X e bytes

e = m-p (Frame Size) bits

DR NISHANT JAIN,
MITS-DU GWLIOR

PAGE TABLE ENTRY



Page Table Entry Format

DR NISHANT JAIN,
MITS-DU GWLIOR

• **Frame Number** – It gives the frame number in which the current page you are looking for is present. The number of bits required depends on the number of frames. Frame bit is also known as address translation bit.

Number of bits for frame = Size of physical memory / Frame Size

• **Present/Absent Bit:** Present or absent bit says whether a particular page you are looking for is present or absent. In case it is not present, that is called Page Fault. It is set to 0 if the corresponding page is not in memory. Used to control page faults by the operating system to support virtual memory. Sometimes this bit is also known as a **valid/invalid** bit.

• **Protection Bit:** The protection bit says what kind of protection you want on that page. So, these bits are for the protection of the page frame (read, write, etc).

• **Referenced Bit:** Referenced bit will say whether this page has been referred to in the last clock cycle or not. It is set to 1 by hardware when the page is accessed.

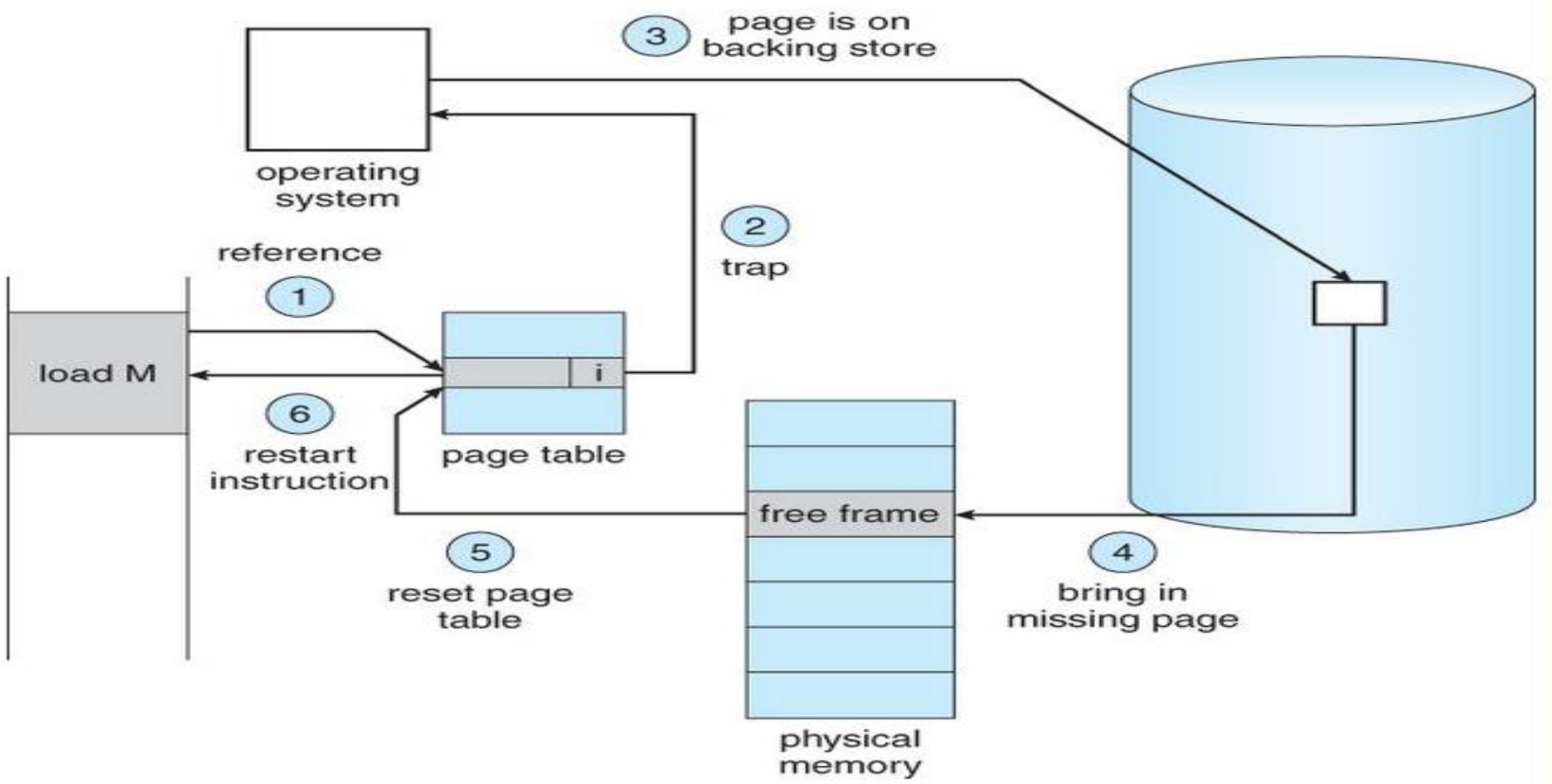
•**Caching Enabled/Disabled:** Sometimes we need fresh data. Let us say the user is typing some information from the keyboard and your program should run according to the input given by the user. In that case, the information will come into the main memory. Therefore main memory contains the latest information which is typed by the user. Now if you try to put that page in the cache, that cache will show the old information. So whenever freshness is required, we don't want to go for caching or many levels of memory. The information present in the closest level to the CPU and the information present in the closest level to the user might be different. So we want the information to be consistent, which means whatever information the user has given, the CPU should be able to see it as first as possible. That is the reason we want to disable caching. So, this bit **enables or disables** caching of the page.

•**Modified Bit:** Modified bit says whether the page has been modified or not. Modified means sometimes you might try to write something onto the page. If a page is modified, then whenever you should replace that page with some other page, then the modified information should be kept on the hard disk or it has to be written back or it has to be saved back. It is set to 1 by hardware on the write-access to a page which is used to avoid writing when swapped out. Sometimes this modified bit is also called the **Dirty bit**.

PAGE FAULT

DR NISHANT JAIN,
MITS-DU GWLIOR

A page fault occurs when a program attempts to access data or code that is in its address space, but is not currently located in the system RAM. So when page fault occurs then following sequence of events happens :



DR NISHANT JAIN,
MITS-DU GWLIOR

IMPORTANT FORMULAE:

For Main Memory-

- Physical Address Space = Size of main memory
- Size of main memory = Total number of frames x Page size
- Frame size = Page size
- If number of frames in main memory = 2^X, then number of bits in frame number = X bits
- If Page size = 2^X Bytes, then number of bits in page offset = X bits
- If size of main memory = 2^X Bytes, then number of bits in physical address = X bits

DR NISHANT JAIN,
MITS-DU GWLIOR

IMPORTANT FORMULAE:

For Process-

- Virtual Address Space = Size of process
- Number of pages the process is divided = Process size / Page size
- If process size = 2^X bytes, then number of bits in virtual address space = X bits

DR NISHANT JAIN,
MITS-DU GWLIOR

IMPORTANT FORMULAE:

For Page Table-

- Size of page table = Number of entries in page table x Page table entry size
- Number of entries in pages table = Number of pages the process is divided
- Page table entry size = Number of bits in frame number + Number of bits used for optional fields if any

DR NISHANT JAIN,
MITS-DU GWLIOR

NOTE:

- In general, if the given address consists of ‘n’ bits, then using ‘n’ bits, 2^n locations are possible.
- Then, size of memory = 2^n x Size of one location.
- If the memory is byte-addressable, then size of one location = 1 byte.
- Thus, size of memory = 2^n bytes.
- If the memory is word-addressable where 1 word = m bytes, then size of one location = m bytes.
- Thus, size of memory = 2^n x m bytes.

DR NISHANT JAIN,
MITS-DU GWLIOR

EXAMPLE-1

Calculate the size of memory if its address consists of 22 bits and the memory is 2-byte addressable.

Solution : We have-

- Number of locations possible with 22 bits = 2^{22} locations
- It is given that the size of one location = 2 bytes

Thus, Size of memory

= 2^{22} x 2 bytes

= 2^{23} bytes

= 8 MB

DR NISHANT JAIN,
MITS-DU GWLIOR

EXAMPLE-2

Calculate the number of bits required in the address for memory having size of 16 GB.

Assume the memory is 4-byte addressable.

Solution-

Let ‘n’ number of bits are required. Then, Size of memory = 2^n x 4 bytes.

Since, the given memory has size of 16 GB, so we have-

2^n x 4 bytes = 16 GB

2^n x 4 = 16 G

2^n x 2^2 = 2^{34}

2^n = 2^{32}

∴ n = 32 bits

Paging Practice Problems

Problem: Consider a logical address space of eight pages of 1024 words each, mapped onto a physical memory of 32 page frames.

(a) **How many bits are in logical address? How many bits represent for the page number and for the offset within a page?**

(b) **How many bits are in physical address? How many bits represent for the page number and for the offset within a page?**

Solution:

a) How many bits are in the logical address?

The logical address is split into two parts: the page address and then the offset.

The page address must be able to reference 8 (or 2^3) distinct pages. This requires 3 address bits.

The offset must be able to reference 1,024 (or 2^10) distinct words on each page. This requires 10 address bits.

Therefore, 13 bits are required for the logical address.

Paging Practice Problems

Problem: Consider a logical address space of eight pages of 1024 words each, mapped onto a physical memory of 32 page frames.

(a) How many bits are in logical address? How many bits represent for the page number and for the offset within a page?

(b) How many bits are in physical address? How many bits represent for the page number and for the offset within a page?

Solution:

b) How many bits are in the physical address?

The physical address is also split into two parts: the frame address and then the offset.

The number of frames is 32 (or 2^5), so that will require 5 bits.

The size of the frame is the same as the size of the page, therefore this will require 10 bits, as described above.

Therefore, 15 bits are required for the physical address.

MAPPING FROM PAGE TABLE TO MAIN MEMORY

DR NISHANT JAIN,
PACE
MITS-DU GWLIOR

Dedicated registers can be used to implement the page table in hardware.

However, using a register for the page table is only useful if the page table is tiny. We can employ TLB (translation look-aside buffer), a particular, tiny, fast look-up hardware cache if the page table has a significant number of entries.

- **The TLB is a high-speed, associative memory.**
- **LB entries are made up of two parts: a value and a tag.**
- **When this memory is accessed, an item is compared to all tags at the same time.**
- **If the object is located, the value associated with it is returned.**

PAGE REPLACEMENT ALGORITHM

DR. NISHANT JAIN,
PAGE
MITS-DU GWLIOR

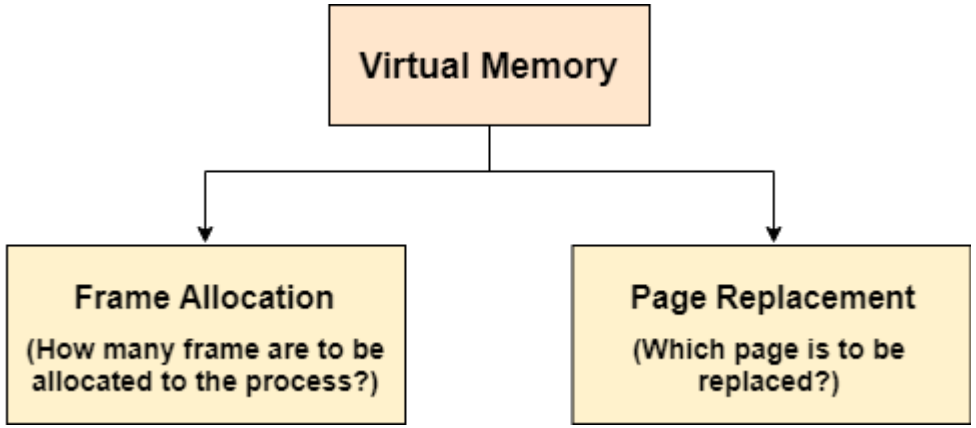
Paging in Operating Systems

Virtual Memory in Operating Systems

Demand Paging

Virtual Memory in Operating Systems

DR NISHANT JAIN,
PACE,
MITS-DU GWLIOR



Frame Allocation Constraints

- The Frames that can be allocated cannot be greater than total number of frames.
- Each process should be given a set minimum amount of frames.
- When fewer frames are allocated then the page fault ration increases and the process execution becomes less efficient
- There ought to be sufficient frames to accommodate all the many pages that a single instruction may refer to

FRAME ALLOCATION

DR NISHANT JAIN,
PACE,
MITS-DU GWLIOR

There are three types of Frame Allocation Algorithms in Operating Systems.

They are:

1) Equal Frame Allocation Algorithms

2) Proportionate Frame Allocation Algorithms

3) Priority Frame Allocation Algorithms

PAGE REPLACEMENT ALGORITHMS

DR NISHANT JAIN,
PACE,
MITS-DU GWLIOR

There are three types of Page Replacement Algorithms. They are:

- Optimal Page Replacement Algorithm
- First In First Out Page Replacement Algorithm
- Least Recently Used (LRU) Page Replacement Algorithm

Example

DR NISHANT JAIN,
PAGE
MITS-DU GWLIOR

Consider the reference string 6, 1, 1, 2, 0, 3, 4, 6, 0, 2, 1, 2, 1, 2, 0, 3, 2, 1, 2, 0 for a memory with three frames and calculate number of page faults by using FIFO (First In First Out) Page replacement algorithms.

6, 1, 1, 2, 0, 3, 4, 6, 0, 2, 1, 2, 1, 2, 0, 3, 2, 1, 2, 0

Points to Remember

Page Not Found - - - > Page Fault

Page Found - - - > Page Hit

Reference String:

S. no	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
F3				2	2	2	4	4	4	2	2	2	2	2	2	2	2	2	2	0
F2		1	1	1	1	3	3	3	0	0	0	0	0	0	0	3	3	3	3	3
F1	6	6	6	6	0	0	0	6	6	6	1	1	1	1	1	1	1	1	1	1
Hit (H)/ Fault (F)	F	F	H	F	F	F	F	F	F	F	F	H	H	H	H	F	H	H	H	F

Number of Page Hits = 8
Number of Page Faults = 12
The Ratio of Page Hit to the Page Fault = 8 : 12 - - - > 2 : 3 - - - > 0.66
The Page Hit Percentage = 8 *100 / 20 = 40%
The Page Fault Percentage = 100 - Page Hit Percentage = 100 - 40 = 60%

Example

DR NISHANT JAIN,
PACE
MITS-DU GWLIOR

Consider the reference string 6, 1, 1, 2, 0, 3, 4, 6, 0, 2, 1, 2, 1, 2, 0, 3, 2, 1, 2, 0 for a memory with three frames and calculate number of page faults by using LRU (Least Recently Used) Page replacement algorithms.

Points to Remember

Page Not Found - - - > Page Fault

Page Found - - - > Page Hit

Reference String:

6, 1, 1, 2, 0, 3, 4, 6, 0, 2, 1, 2, 1, 2, 0, 3, 2, 1, 2, 0

S. no	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
F3				2	2	2	4	4	4	2	2	2	2	2	2	2	2	2	2	2
F2		1	1	1	1	3	3	3	0	0	0	0	0	0	0	0	0	1	1	1
F1	6	6	6	6	0	0	0	6	6	6	1	1	1	1	1	3	3	3	3	0
Hit (H)/ Fault (F)	F	F	H	F	F	F	F	F	F	F	F	H	H	H	H	F	H	F	H	F

Number of Page Hits = 7
Number of Page Faults = 13
The Ratio of Page Hit to the Page Fault = 7 : 12 - - - > 0.5833 : 1
The Page Hit Percentage = 7 * 100 / 20 = 35%
The Page Fault Percentage = 100 - Page Hit Percentage = 100 - 35 = 65%



Segmentation

Memory-management scheme that supports user view of memory

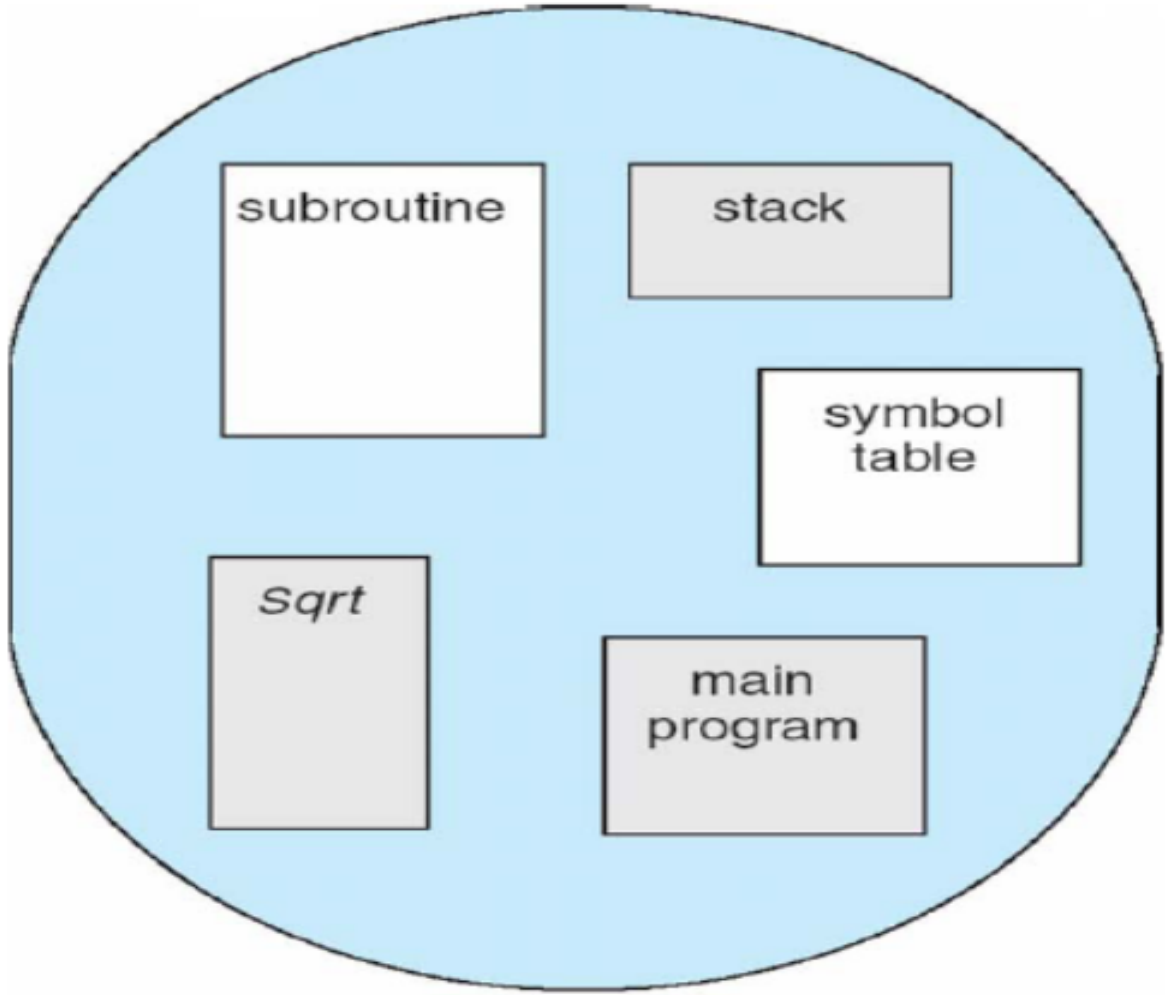
A program is a collection of segments

A segment is a logical unit such as:

- main program
- procedure
- function
- method
- object
- local variables, global variables
- common block
- stack
- symbol table
- arrays



User's View of a Program

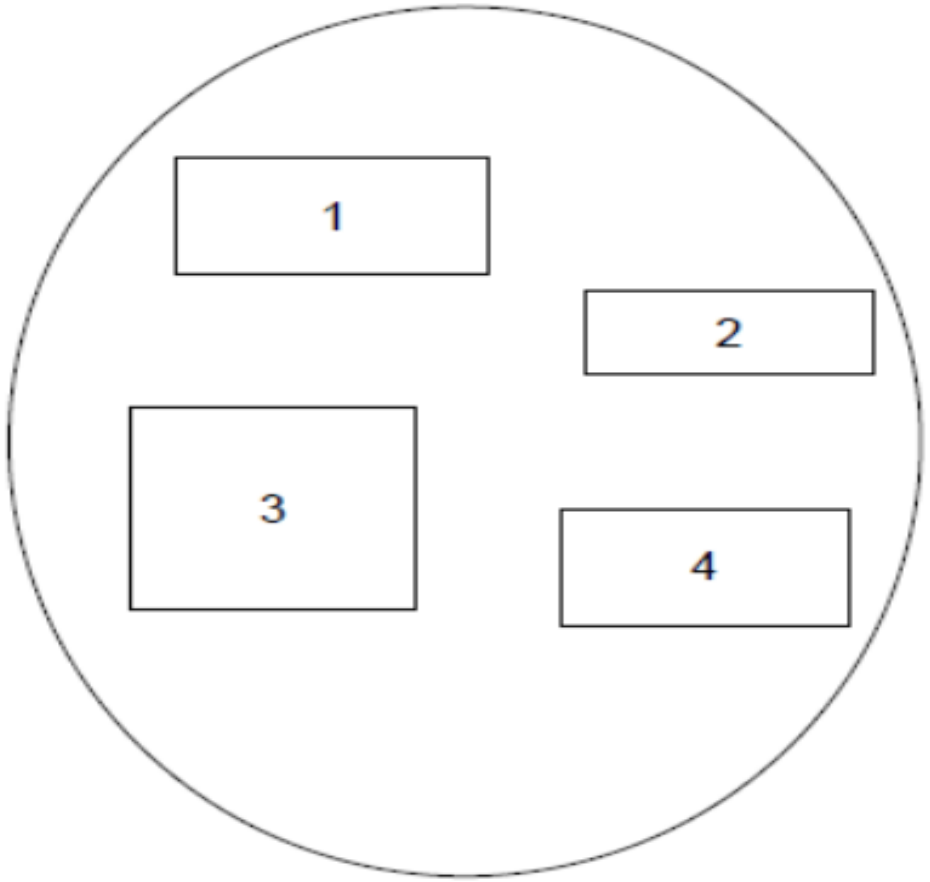


logical address

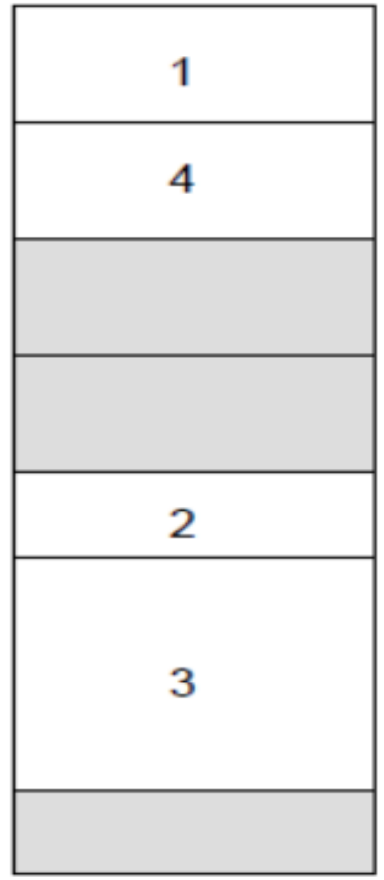


Logical View of Segmentation

DR NISHANT JAIN,
MITS-DU GWLIOR



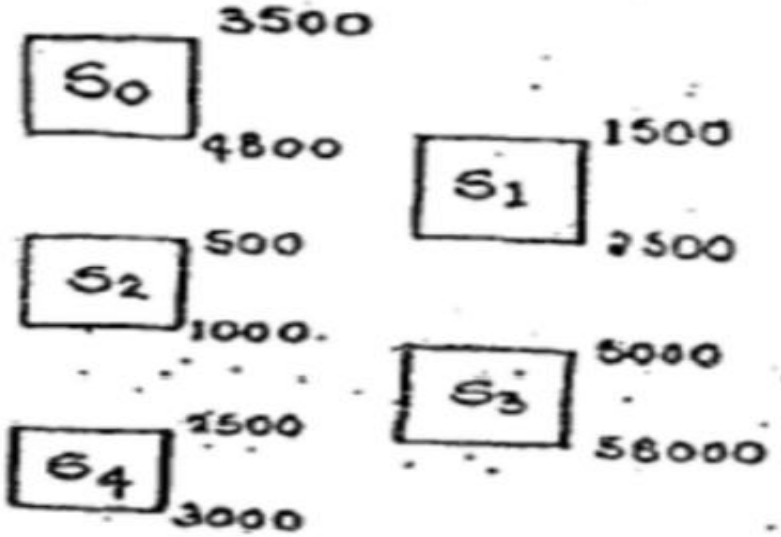
user space

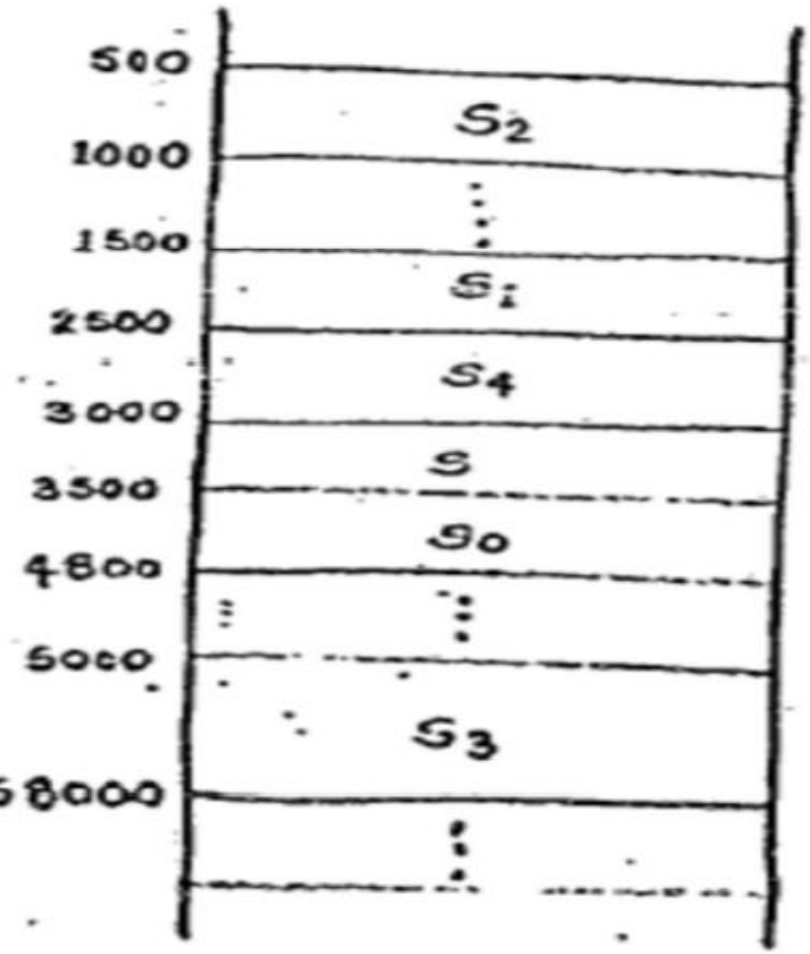


physical memory space

Segmentation

Paging doesn't preserve user's view of memory allocation



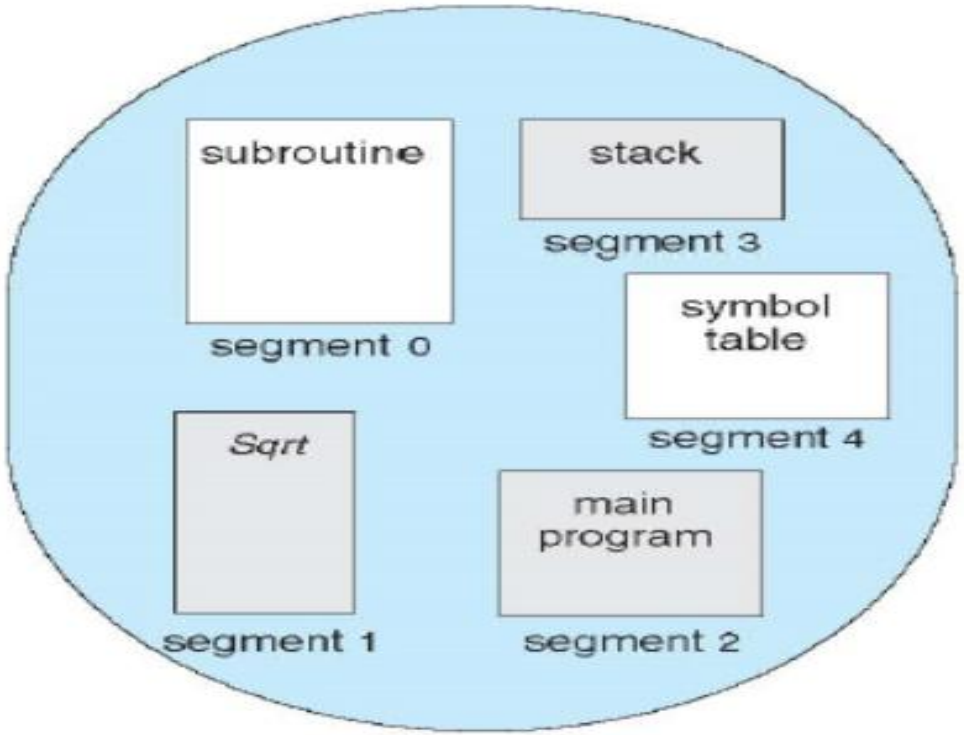


Segment is a piece of code.

'program, divided into unequal sizes is called Segment (for equal size is called pages)).

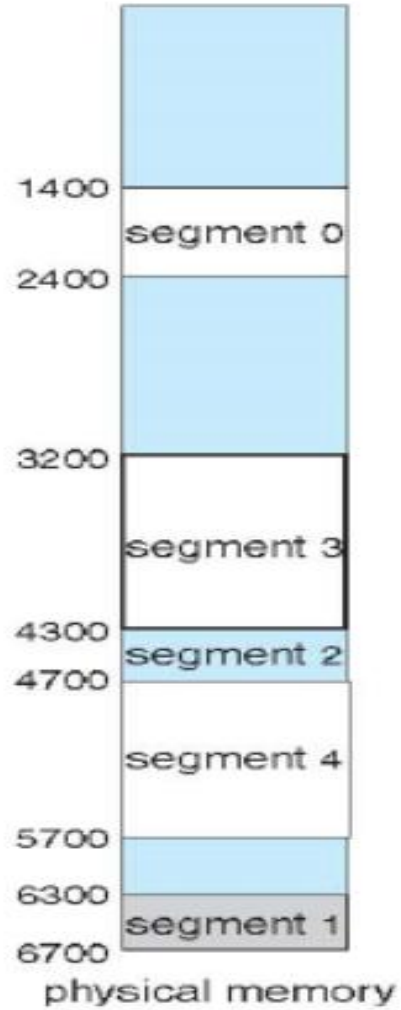


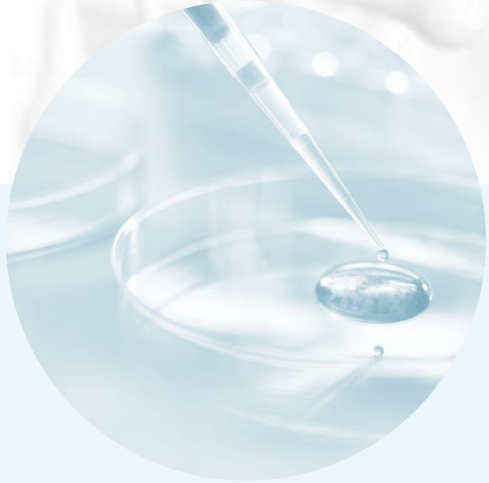
Example of Segmentation



	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table





THANK YOU